Log-Structured File Systems

Design Principles, Recovery, Cleaning and a Sprite LFS Case Study

Prashant Jadav

DEPSTAR – CHARUSAT
Fundamentals of Operating System Design (CEUC301)

Learning Objectives

By the end of this lecture, students should be able to:

1. Explain the motivation and core idea of a Log-Structured File System (LFS).
2. Describe sequential log writing, segment organization and inode-map updates.
3. Trace file updates, crash recovery and checkpointing.
4. Explain why segment cleaning is necessary and how it works.
5. Evaluate LFS advantages, costs and workload suitability.
6. Analyze a worked Sprite LFS case study and compare LFS with a conventional file system.

Prerequisites: inodes, directories, disk blocks, caching, file-system metadata and crash recovery.

Motivation: Why Reconsider Disk Writes?

Traditional file systems often update data and metadata at different locations:

- A file's data blocks may be far from its inode.
- Directory blocks, allocation maps and inode tables may require separate writes.
- Small random writes can incur seek and rotational latency on magnetic disks.
- Metadata updates may generate many small I/O operations.

Observation

Many workloads accumulate changes in memory before writing them. If the file system writes these changes together in large sequential batches, it can reduce the cost of scattered writes.

LFS design goal: turn most writes into sequential appends to a log, rather than overwriting blocks in place.

What Is a Log-Structured File System?

Definition

A **Log-Structured File System (LFS)** writes new versions of file data and metadata sequentially into a log-like structure. Updated objects are generally appended at new locations; older versions become obsolete.

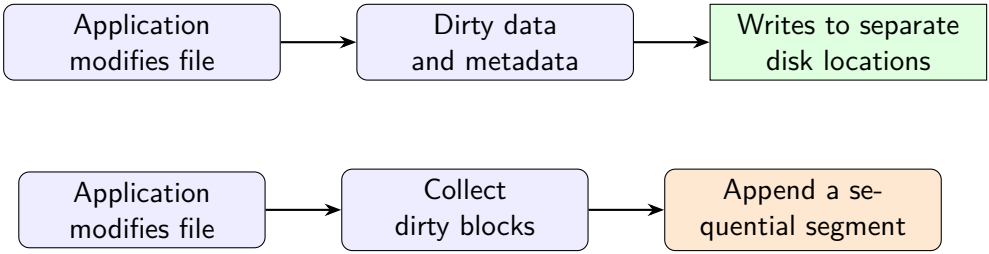
Core principles

1. Buffer modifications in memory.
2. Collect dirty data and metadata into a segment.
3. Write the segment sequentially to the storage device.
4. Update metadata that maps file identifiers to their latest locations.
5. Reclaim space occupied by obsolete versions using segment cleaning.

Important

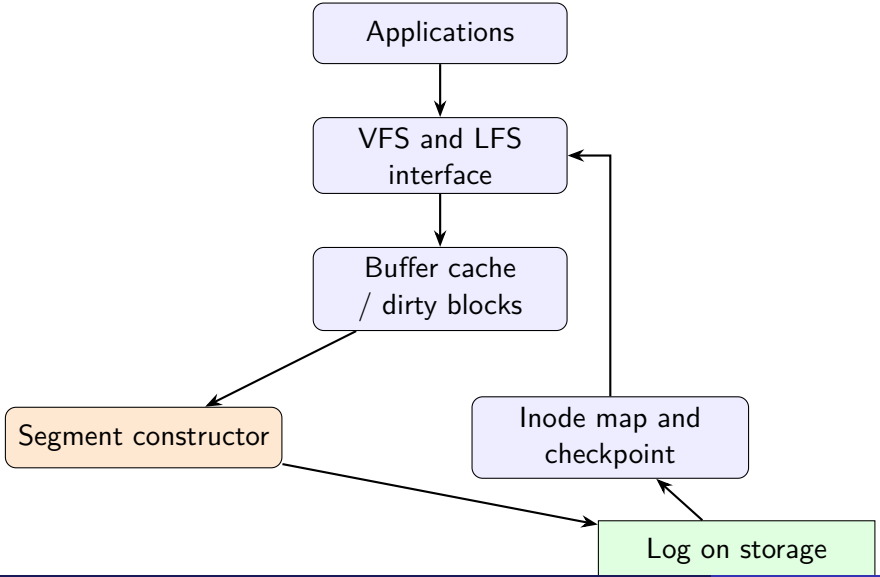
An LFS is not merely a text log of file operations. Its log contains file-system blocks and metadata needed to locate current versions of files.

Conventional Updates vs. LFS Updates



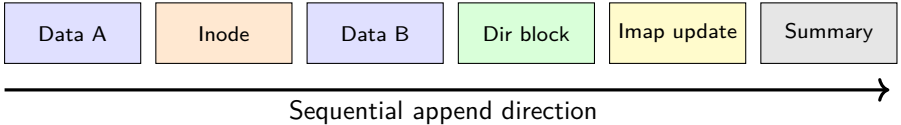
- Conventional in-place update: data and metadata can trigger separate, nonsequential writes.
- LFS: related dirty blocks are assembled into a larger sequential write.
- LFS still performs reads and may perform cleaning I/O; not every operation is sequential.

High-Level LFS Architecture



The Log and Segment Organization

The log is divided into fixed-size regions called **segments**. A segment is a unit of writing and often of cleaning.



- Segment contents can include file data, inode blocks, directory blocks, inode-map information and a summary.
- Segment summaries help identify the blocks represented in a segment.
- The active log tail is where new segments are appended.
- When the tail reaches the end of available space, the system selects another free segment.

How Does an LFS Update a File?

Suppose a file `notes.txt` has an existing data block at location P and is modified.

1. The application changes bytes in memory.
2. The buffer cache marks the affected data block dirty.
3. The segment constructor gathers the new data block and related dirty metadata.
4. LFS appends the new data block at location Q and writes updated inode information.
5. The inode map is updated so the file's inode points to the new version.
6. The old block at P is no longer the current version and can eventually be reclaimed.

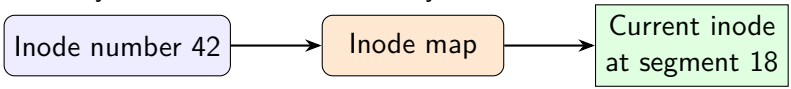
$$\text{old version at } P \quad \longrightarrow \quad \text{new version at } Q$$

Consequence

The update generally does not overwrite P immediately. The newest valid location is found through the file-system's metadata.

Inode Map: Finding the Current Version

In an LFS, an inode can move whenever its updated version is written to a new location. The file system therefore needs a way to locate the latest inode.

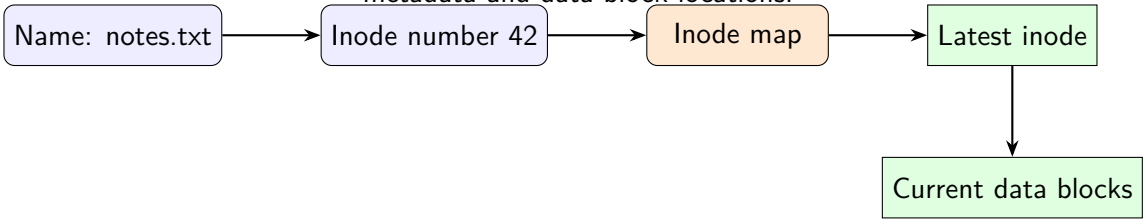


When the inode is updated, a new inode copy is appended and the inode map is changed to point to that copy.

- The inode map maps inode numbers to the disk addresses of current inode versions.
- Implementations may organize or cache the map in different ways.
- Checkpoint information helps recover a consistent inode-map state after a crash.

Directories, Inodes and Data Blocks

A directory entry typically maps a name to an inode number; the inode then identifies file metadata and data-block locations.



- Updating a file may require new data blocks and a new inode version.
- Creating or renaming a file may also update directory blocks.
- Those changes can be appended together in a segment, but references must remain recoverable and consistent.

Crash Recovery: Checkpoints and Roll-Forward

LFS recovery commonly combines a **checkpoint** with a scan of log records written after that checkpoint.

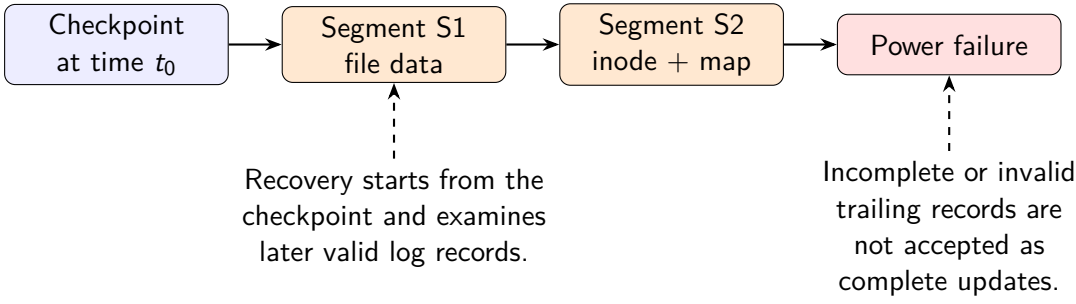
1. Locate a valid checkpoint region.
2. Restore key state, such as the inode-map state and log position.
3. Scan forward through later log segments.
4. Validate segment summaries and records according to the implementation.
5. Incorporate complete, valid updates that belong to the recoverable log.
6. Resume normal operation at a valid log tail.

Why checkpoints matter

Without checkpoints, recovery might need to scan the entire log. Checkpoints limit the amount of log that typically needs to be examined.

Caution: Exact recovery rules depend on the LFS implementation and its write-ordering and consistency protocol.

Crash-Recovery Timeline



A checkpoint is not necessarily a copy of every file's contents. It is a recoverable summary of file-system state; later log records may contain additional changes.

Why Does LFS Need Segment Cleaning?

Since updates append new versions, the log eventually fills with a mixture of live and obsolete blocks.

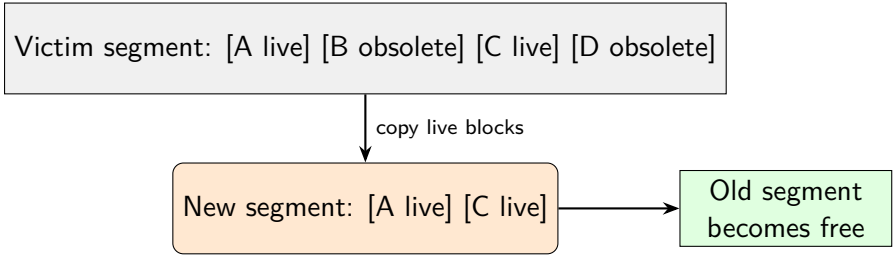
- **Live block:** contains the current version of data or metadata.
- **Obsolete block:** an older version no longer referenced by current metadata.
- **Free segment:** available for new sequential writes.

Problem

Appending is efficient while free segments are available. To continue operating, LFS must reclaim segments containing obsolete blocks.

Segment cleaning copies live blocks out of a selected segment, updates references as needed, and frees the old segment for reuse.

Segment Cleaning: A Worked Diagram



1. Choose a victim segment.
2. Determine which blocks are still live.
3. Copy live blocks to a new segment.
4. Update metadata references to copied blocks.
5. Mark the victim segment as free.

Trade-off

Cleaning consumes I/O. If most blocks in a victim segment are live, little space is reclaimed.

Cleaning Cost and Segment Utilization

Let a victim segment have N blocks, of which a fraction u are live.

$$\text{Live blocks copied} = uN$$

$$\text{Blocks reclaimed} = (1-u)N$$

Ignoring metadata and other overhead, a simple estimate of total I/O is:

$$\text{I/O cost per reclaimed block} \approx \frac{N + uN}{(1-u)N} = \frac{1+u}{1-u}$$

Here, N approximates the cost of reading the victim segment and uN the cost of writing its live blocks. This is a simplified model, not a universal performance formula.

Live fraction u	Approximate I/O per reclaimed block
0.20	1.50
0.50	3.00
0.80	9.00

Interpretation: cleaning is more efficient when selected segments contain a high fraction of

LFS Performance: Benefits and Costs

Potential benefits

- Large sequential writes.
- Efficient batching of small updates.
- Data and metadata can be written together.
- Checkpoint-based crash recovery.
- Particularly attractive for write-heavy workloads on seek-sensitive storage.

Costs and challenges

- Segment cleaning adds background I/O.
- Cleaning can create write amplification.
- Free-space management is essential.
- Random reads may require metadata lookups and nonsequential access.
- Performance can degrade when the disk is nearly full.

LFS vs. Journaling File Systems

These designs both use sequential records in some form, but their primary purposes differ.

Aspect	Log-Structured FS	Journaling FS
Main idea	Append new versions of data and metadata into a log	Record changes in a journal before or while updating home locations
Normal data placement	Updated blocks generally receive new log locations	Home blocks are typically updated in place after journaling, depending on mode
Space reclamation	Segment cleaning is fundamental	Journal space is reclaimed after checkpointing; ordinary file blocks are managed separately
Write pattern	Designed to make most writes sequential	Journal writes are sequential, but checkpoint writes may be scattered
Examples	Sprite LFS; later log-structured designs	ext4 with JBD2; NTFS journaling metadata

Key distinction:

Case Study: Sprite Log-Structured File System

Sprite LFS was developed as a research system to explore log-structured storage and efficient write performance.

Problem context

Workstations generate many small file and metadata updates. On disks where seeks are expensive, writing each update to its conventional location can make the workload I/O-bound.

Design approach

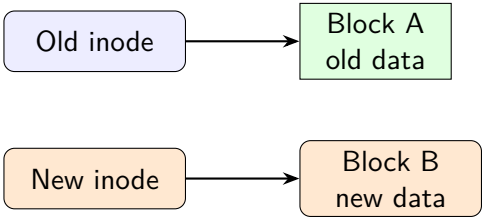
- Accumulate dirty file data and metadata in memory.
- Pack updates into large segments.
- Append segments sequentially to the log.
- Use an inode map and checkpoint regions to locate current file-system state.
- Use segment cleaning to reclaim space occupied by old versions.

Research lesson: sequential writes can improve write throughput, but the system must manage live data, cleaning costs and recovery carefully.

Sprite LFS Case Study: Updating a File

Assume a user edits `report.txt`, whose current inode points to data block A.

1. The changed data is buffered in memory.
2. LFS appends the new data block B into a segment.
3. The updated inode is appended, now referring to B.
4. The inode map is updated to point to the new inode location.
5. A checkpoint later captures a recoverable state.



Inode map now resolves the file to the new inode version.

Sprite LFS Case Study: Crash Scenarios

Crash point	What may be on disk?	Recovery reasoning
Before new segment is durable	Old version may remain current; new update may be absent	Start from checkpoint and accept only valid recoverable records
After new segment, before checkpoint	New blocks may exist beyond checkpoint	Roll forward through valid post-checkpoint log records
During segment write	Trailing segment may be incomplete	Validate segment information; do not accept incomplete records as complete updates
After checkpoint is durable	Checkpoint describes a newer recoverable state	Resume from that checkpoint and scan later records if present

Implementation caveat

The exact durability boundary depends on the system's write protocol and storage guarantees. The table describes the conceptual recovery model, not a promise that every arbitrary write ordering is safe

Worked Numerical Example: Cleaning a Segment

Suppose a segment contains 1,000 blocks. During cleaning:

- 300 blocks are live.
- 700 blocks are obsolete.
- All 300 live blocks must be copied to another segment.

Ignoring metadata overhead:

$$N = 1000, \qquad u = 0.30$$

$$\text{Read I/O} \approx 1000 \text{ blocks}$$

$$\text{Write I/O} \approx 300 \text{ blocks}$$

$$\text{Space reclaimed} = 700 \text{ blocks}$$

$$\text{I/O per reclaimed block} \approx \frac{1000 + 300}{700} \approx 1.86$$

Interpretation: 1,300 block-I/O equivalents are spent to recover 700 blocks of free capacity under this simplified accounting model. Actual costs depend on segment selection, caching, metadata and device behavior.

When Is LFS a Good Fit?

Potentially suitable

- Write-heavy workloads with many small updates.
- Systems where sequential write bandwidth is valuable.
- Workloads that can batch modifications effectively.
- Storage designs that can manage cleaning and free space efficiently.

Potential concerns

- Nearly full storage, where free segments are scarce.
- Workloads with high live-data fractions in candidate cleaning segments.
- Workloads dominated by random reads, depending on layout and caching.
- Systems where cleaning overhead competes with foreground I/O.

Modern context

The term "log-structured" also appears in flash storage, key-value stores and log-structured merge trees. These systems share append-oriented ideas but are not all the same as a general-purpose LFS.

Common Misconceptions

1. **LFS stores only text logs.** False: its log contains file-system data and metadata blocks.
2. **Old blocks are overwritten immediately.** Generally false: new versions are appended and old versions become obsolete.
3. **Sequential writes eliminate all random I/O.** False: reads, metadata lookups and cleaning can involve nonsequential access.
4. **Cleaning is optional.** False for a finite-capacity append-oriented LFS; space must eventually be reclaimed.
5. **LFS and journaling file systems are identical.** False: their primary on-disk organization and update strategy differ.
6. **A checkpoint contains all file data.** False: it is a recoverable summary of state, not necessarily a full data copy.

Summary

- LFS appends new versions of file data and metadata to a log.
- Sequential segment writes can reduce seek overhead and improve write throughput.
- An inode map helps locate current inode versions.
- Checkpoints and roll-forward recovery reconstruct state after a crash.
- Segment cleaning copies live blocks and reclaims obsolete space.
- Cleaning efficiency depends strongly on segment utilization.
- LFS trades sequential write efficiency for additional metadata management and cleaning complexity.

Takeaway

The central LFS idea is to make updates sequential; the central LFS challenge is to reclaim space without losing the performance benefit.

Review and Discussion Questions

1. Why can sequential writes outperform scattered writes on a magnetic disk?
2. What is the role of the inode map in LFS?
3. Explain checkpointing and roll-forward recovery.
4. Why does LFS require segment cleaning?
5. A 1,000-block segment contains 250 live blocks. How many blocks are immediately reclaimable after copying the live blocks?
6. Compare LFS with a metadata-journaling file system.
7. In the Sprite LFS case study, trace the steps involved when a file is updated.
8. How might a nearly full LFS affect foreground write performance?

Suggested References



R. H. Rosenblum and J. K. Ousterhout,
"The Design and Implementation of a Log-Structured File System,"
ACM Transactions on Computer Systems, vol. 10, no. 1, 1992, pp. 26–52.



R. Arpaci-Dusseau and A. Arpaci-Dusseau,
Operating Systems: Three Easy Pieces,
File-System Implementation and Crash-Consistency chapters.



A. Silberschatz, P. B. Galvin and G. Gagne,
Operating System Concepts,
File-System Implementation and Recovery topics.

Teaching note: Use the Sprite LFS paper for implementation details and the textbook chapters for introductory file-system concepts.